北京交通大学

Software Defined Multicast Using Segment Routing in LEO Satellite Networks

低轨卫星网络中基于段路由的软件定义组播

汇报人：蒋凯 万李裔 陈浙豫 程新越 任昶宇

2026.11.2

目录

01 研究背景

02 系统架构

03 算法设计

04 实验评估

05 总结与思考

第一部分

研究背景

视频分发： LEO 星座的机会与现有链路的浪费

组播天然适配“一对多”的视频业务：同一份内容只需在树上各边各传一次

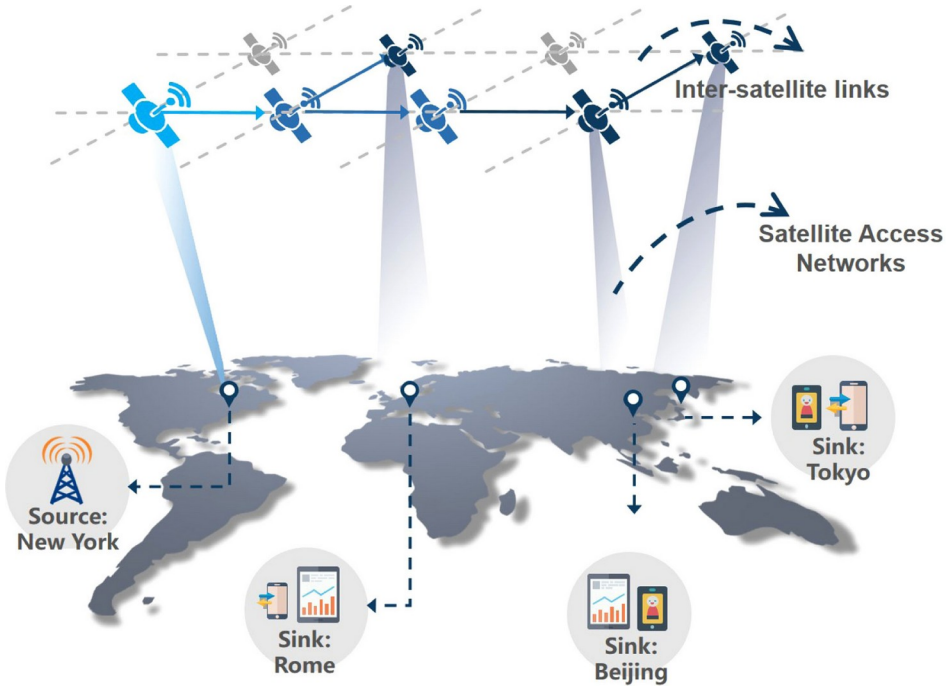


Fig. 1 源（纽约）经 ISL 组播树把视频送达罗马 / 北京 / 东京三个宿，接入网与星间链路均只承载一份流量

- **星座规模** Starlink 到 2022 年 3 月已发射 2,200+ 颗卫星， OneWeb 、Iridium-NEXT 同期推进
- **带宽浪费** CDN 用应用层组播（ALM）降低骨干流量，但“ CDN → 用户”仍是单播，城域网与接入网被重复流量占满
- **LEO 的机会** 把内容改为在 SAN 与 ISL 上组播，一份视频在每条链路上只传一次

2,200+	P × Q	4
2022.3 在轨 Starlink 卫星（论文引用）	平面数 × 每平面卫星数，全同构	每颗卫星的邻接 ISL 数

单播把同一视频在每条链路上重复 N 次；组播只在树的分叉处复制，链路收益随组规模线性放大。

论文的主张

把内容分发从“应用层组播 + 单播”升级为“网络层组播”，在 SAN 与 ISL 上同时受益

对全球分布的 CDN 边缘节点尤其有效，且收益随组规模放大

三段研究空白： IPM 、SDM 未进卫星、卫星侧未做组播路由

结论： SDM 组播路由在 LEO 星座中“从未被利用”，这正是本文的切口

IP 组播（IPM）

PIM / mLDP 依赖逐组树状态， BIER 仍需“报文头 + 精简转发表”双份状态；分布式决策拿不到全局信息

→ 带宽节省有限，且组播状态随组数线性增长，星上 TCAM 很快耗尽

SDM 研究（地面网）

综述与算法均以 mesh 拓扑的地面光网络为假设，未利用 ISL 的矩形栅格拓扑来优化性能与可扩展性

→ 无法利用栅格做“直角最短路 + Steiner 点”的结构性优化

卫星 SDN 组播

已有工作只关注 SD-STIN 架构、单播、时变图节能组播、拍卖式流量卸载；少数 LEO SDM 尝试仅做树构建

→ 组播路由（树构建 + 转发 + 恢复）整体缺位，段路由协议从未设计

空白点很清晰：没有人把集中式组播树的全局视野，与卫星网络规整的栅格拓扑、段路由的极简报文头结合起来。

组播技术演进与本文位置

CDN / ALM

减轻骨干流量，接入网仍单播

IP 组播IPM

分布式、逐组状态、扩展性差

SDM （地面 mesh ）

集中式优化树，未考虑栅格拓扑

SDM + SR in LEO

本文：栅格拓扑 + 段路由 + 树管理

五个必须回答的工程约束，决定了协议的全部设计取舍

每条挑战都对应论文中的一个具体机制——不是开放式问题，而是被逐一消解的设计约束

01 频繁切换（Handover）

卫星高速移动，用户不断更换附着卫星；传统切换方案与本文协议、场景不匹配

02 星载存储受限

路由 / 交换使用有限的高速 TCAM，传统组播的逐组、逐路径状态难以扩展

03 ISL 负载不均

多条会话共存时局部节点 / 链路成为热点，需要在不增大报文头的前提下均衡

04 成员频繁更新

组播组的加入 / 离开需要快速、灵活的成员管理，且不能破坏树的最优性

05 故障与重传风暴

故障后若向全网重传，源会被大量 ACK 与重传流量淹没

论文的一对一回应

切换	地面栅格 + 地标逻辑地址：用户地址不变，卫星换位
存储	只有源卫星维护转发表；中继仅按标签转发
负载	段内 ECMP：在包围盒内随机走等代价斜边
成员	O(log n) 树恢复：边替换 / 最小覆盖重连子树
故障	局部重传：只向丢包的子树（local root）重发

1.4 本文贡献

论文的贡献声明（contributions，

C1 首个面向宽带 LEO 星座的 SDM 框架

- 以 RST 与 RSA 两类直角斯坦纳树构建组播树
- RST 最小化树长 → 最省带宽； RSA 在最短路径约束下最小化树长 → 兼顾时延
- 实验上显著优于传统 CDN 与 IP 组播（SPT）

C2 敏捷、可扩展、低开销的段路由组播协议

- 用地面网格的逻辑地址吸收频繁切换，路由方案在切换后依然有效
- 段内采用 ECMP 感知的单播路由，压缩报文头并实现负载均衡
- 只有源卫星需要转发表，中继卫星无需任何路径状态

C3 快速高效的组播树恢复

- 适应频繁的组成员更新与网络故障
- RST 用边替换、 RSA 用最小覆盖重连断开子树
- 复杂度 O(log n)，百成员规模下单次操作 0.01–1 ms

C4 局部重传（Local Rerouting）

- 引入 local root： 断开子树的根
- 只向“丢包的子树”重传，而非向全体成员重传
- 显著削减重传流量，避免源端被 ACK 淹没

一句话概括：把“全局最优的组播树”放进“极简的段路由报文头”，再用“ O(log n) 的树管理”让它扛住卫星网络的动态性。

第二部分

系统架构

地面控制器计算组播树，卫星依据标签完成转发

控制器负责路由决策与配置；源卫星、段内中继及树成员节点分别执行对应的数据处理。

地面 SDN 控制器

掌握拓扑与源／接收端逻辑地址；计算 RST 或 RSA 。

生成有序段列表，并配置源卫星的转发表。

配置源端

源卫星

接收地面源的数据，

封装段列表为标签栈。

段内中继

读取活动标签，

朝局部目的节点转发。

树成员节点

按子节点复制、更新标签；

有附属用户时交付数据。

地面视频源

地面接收用户

源路由模式下的状态分工

源卫星保存控制器配置的树／段信息。

中继依据活动标签和邻接信息转发。

树成员启动后续段，并向附属用户交付数据。

从建树到交付

计算组播树 → 配置源端 → 封装标签 → 星间转发 → 用户接收

同构星座可由单一机构统一部署，并通过 SDN 控制器集中管理。

P×Q 同构卫星构成四邻接星间网络

轨内 ISL 与轨间 ISL 共同提供上、下、左、右四个方向的相邻连接。



图 2 Walker star 型 LEO 星座

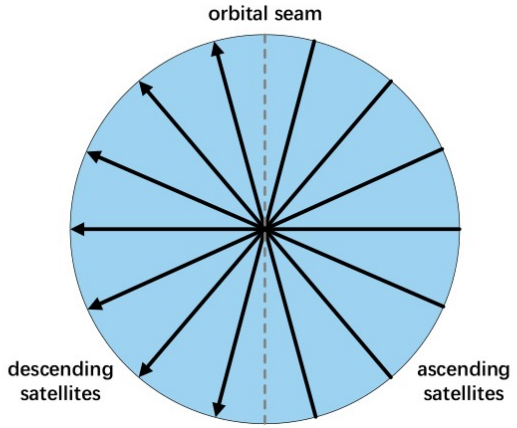


图 3 升轨、降轨与轨道缝

轨道组织

P 个轨道平面，每平面 Q 颗同构卫星，总数为 P×Q。

四邻接连接

上／下邻居位于同一轨道，左／右邻居位于相邻轨道。

ISL 通信方式

星间链路可采用激光或微波通信。

轨道缝

升轨与降轨相对速度高，跨缝 ISL 不建立。

极区处理

极区卫星休眠并关闭路由，非极区形成独立网格。

跨网格通信

两个网格可经地面骨干互联；组播模型研究其中一个网格。

用网格描述空间连接，用时隙描述拓扑变化

组播树构建采用相对位置与跳数；各链路容量相同，拓扑按短时隙进行建模。

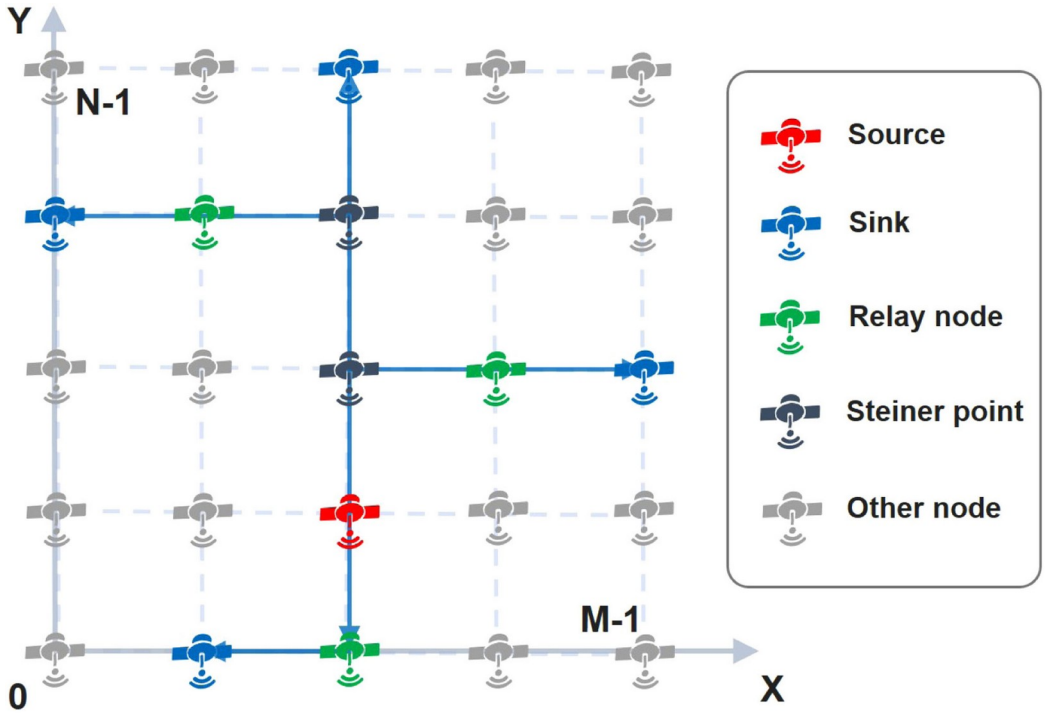


图 4 网格中的源、接收端、中继与 Steiner 点

空间抽象： G(V,E)

卫星映射为二维坐标 (x,y)， ISL 对应边 e。

一个独立网络表示为 M×N 直角网格。

各边带宽容量相同，距离按横、纵方向跳数之和计算。

时间抽象：短时隙

卫星轨道及速度已知，每个时隙的 ISL 拓扑可提前计算。

时隙内用户连接的卫星固定，可据此逐时隙构造组播树。

度量口径：树长用于衡量总链路占用；源至接收端的路径跳数用于衡量时延，二者对应不同的优化目标。

逻辑地址绑定地面区域，切换时更新卫星映射

先按地标的逻辑地址构造组播树，再映射到当前承载相应区域的卫星。

地面区域与地标

卫星投影到带有逻辑地址的地面地标。地面划分为以地标为中心的平行四边形区域，区域内用户由该地标表示。

逻辑树与卫星映射

依据组成员的地标地址构造组播树，再映射为连接当前服务卫星的镜像树；地面源先接入本区域当前的卫星。

切换时更新映射

卫星同步更新关联地标、自身逻辑地址和四个邻居的逻辑地址；用户切换服务卫星后，继续使用其区域的逻辑地址。

同步前提：卫星与用户需要同步时钟。逻辑地址与物理卫星的映射更新，使组播路由能够适应周期性切换。

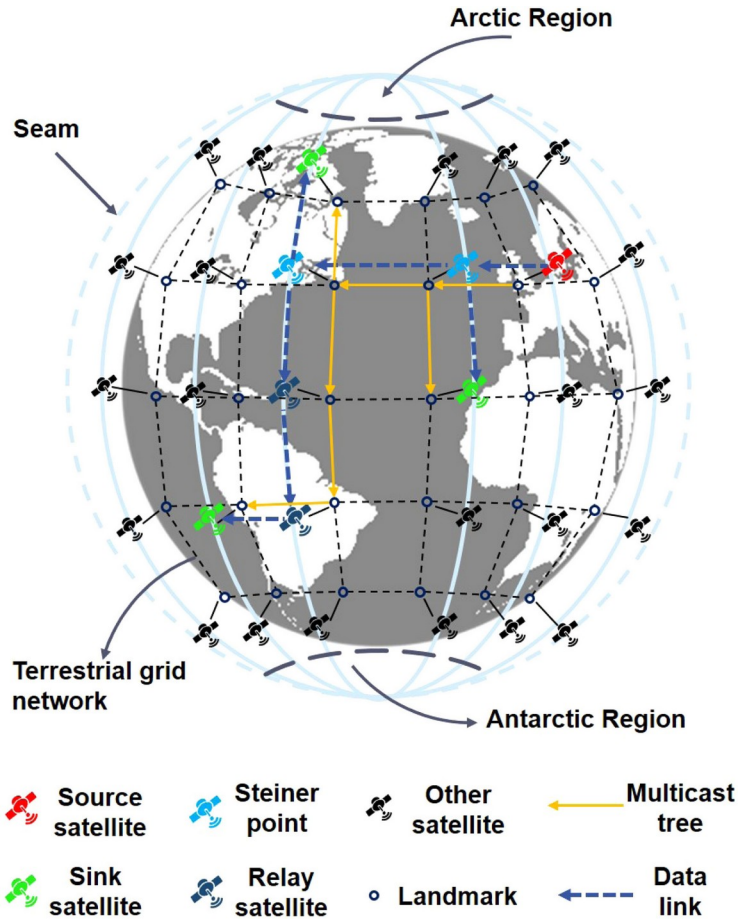


图 6 地面地标、逻辑地址与卫星镜像组播树

第三部分

算法设计

四种树的取舍： RST 最省带宽， RSA 时延最优

论文用一组统一指标比较六种算法：树长（带宽）、平均时延、运行时间、表项压缩、 ECMP 比例

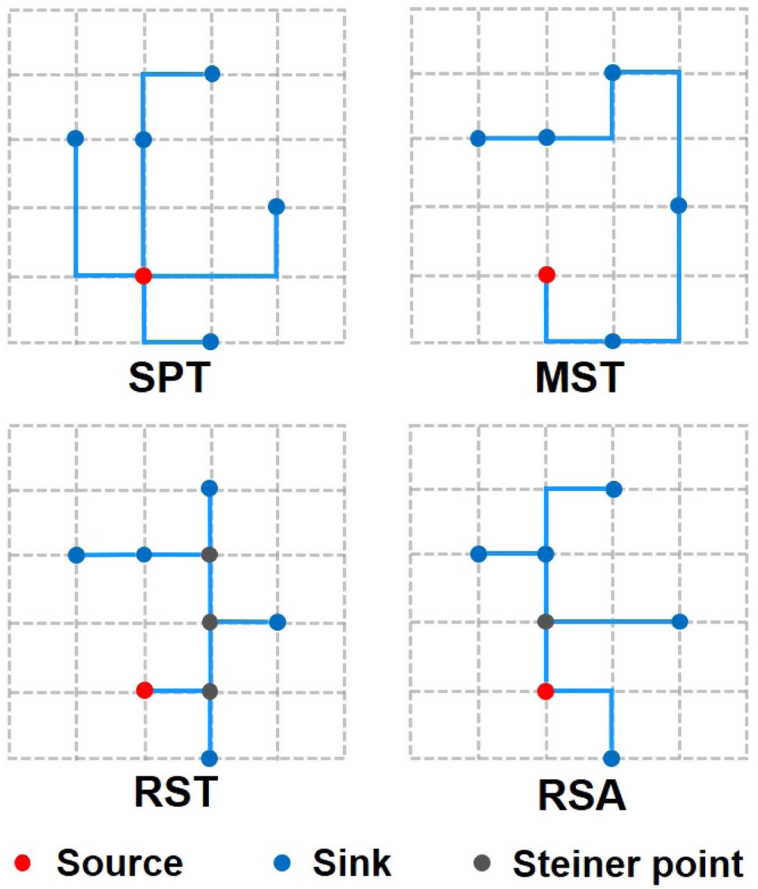


Fig. 5 同一源宿集合上的四种组播树（红 = 源，蓝 = 宿，灰 =Steiner 点）

算法	优化目标	树长（带宽）	源–宿时延
SPT	每条源–宿路径都是最短路径（IPM 采用）	最差	最优
MST	全树边权之和最小	中等	较差
RST	允许引入 Steiner 点，直角总长最小	最优	中等
RSA	在“源到每宿均最短”约束下总长最小	良好	最优

- **Steiner 点** RST / RSA ‘—— SPT / MST 缺少的自由度
- **对比动力** RSA 用“最短路径约束”把时延钉死在地理下界，再在可行域内最小化树长，因此带宽与时延同时接近最优
- **指标径** 树长 → 带宽消耗；平均时延 → 源到各宿路径跳数的均值；两者由不同的树形状主导

代价： RST 与 RSA 均为 NP-complete （文献 [35]，[17]），精确算法在最坏情况下指数时间，必须走多项式启发式。

RST
树长最优

RSA
时延最优

SPT
树长最差

NP-hard
两者均需近似

RST： 把带宽目标变成几何最短，再用“边替换”在多项式时间内逼近

每个 ISL 跳消耗大致相同的带宽 ⇒ 最小直角总长 = 最小 ISL 跳数 = 最小带宽消耗

定义（RST）

在平面上连接终端集合 T 的树：允许经过额外 Steiner 点，只使用水平 / 垂直边，使直角总长最小

对 M×N 栅格而言，就是选一组 ISL，让源与全部宿连通且总跳数最少

为什么它就是带宽最优

栅格拓扑下每跳 ISL 带宽消耗近似相同 → 树长（直角距离）与带宽占用一一对应

因此“几何最短”与“省带宽”是同一个问题的两种说法，且与星座规模无关

算法：生成图 + 边替换（Zhou [39]，本文采用）

先用生成图（spanning graph [40]）产生初始生成树，再对候选“点–边对”做边替换得到更优树

性能：仅比精确算法差约 1%，但快数百倍；建树复杂度 O(n log n)（n 为组规模）

选它的理由：在性能与速度之间取得最佳平衡，且实现简单，便于同时用于建树与树管理

RST 不是从源“生长”出来的：得到树后需把源指定为根，沿树做一次 DFS 确定每个节点的父 / 子关系——这是后续 BFS 排标签栈的前提。

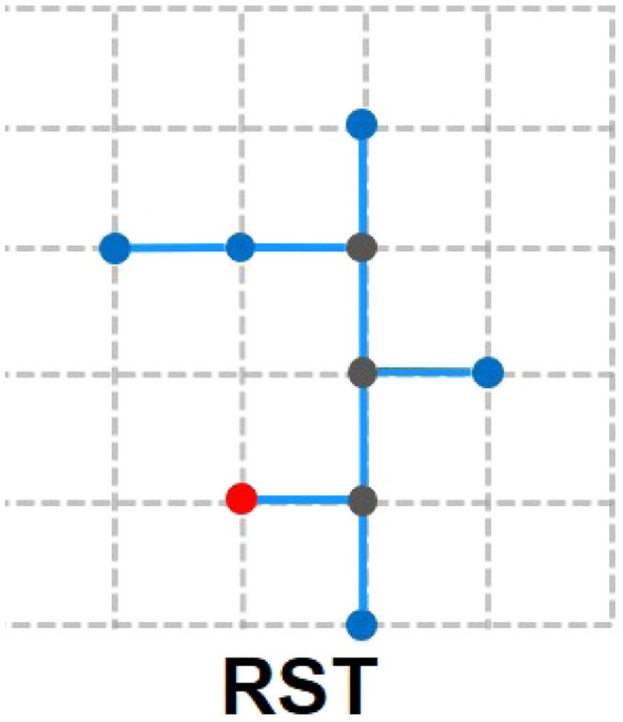


Fig. 5(b) RST：三个 Steiner 点共用一条主干

树长排序： RST < RSA < MST < SPT
RST 的三处分叉均由 Steiner 点承担

RSA：先钉死时延下界，再最小化树长——用最小覆盖做 2- 近似

RSA 是有向树：根到每个终端都必须是直角最短路；在此约束下让总长最小

定义（RSA）

以源 r 为根的有向树： r 到任意终端 t 的路径都是直角最短路径，且全体边长之和最小

与 RST 的唯一区别是多了一条"源到每宿必须最短"的硬约束

双目标含义

带宽：总长最小 → ISL 消耗最少；时延：每宿路径最短 → 端到端延迟被压到几何下界

算法：最小覆盖（minimal cover [42]）—— 2- 近似， O(n log n)

① 从源出发

以源为中心向外扩展

② 多轮覆盖

每一轮覆盖尚未覆盖的宿

③ 合并边

被覆盖的宿共享已建路径

树管理时同样只处理"各子树根"，因此单次恢复为 O(log n)，且最小覆盖天然可避开故障点。

分工建议：带宽是主要矛盾时选 RST；传播时延成为主要矛盾时选 RSA—— 这也是实验部分的最终推荐。

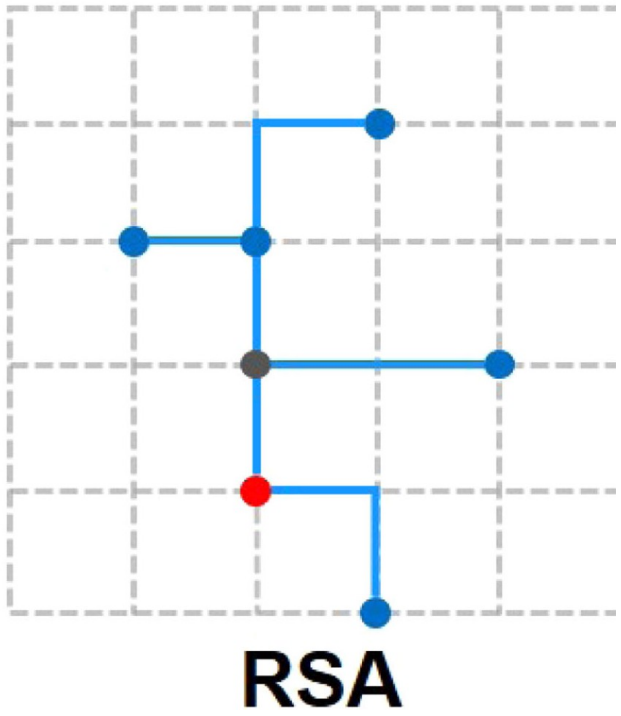


Fig. 5(d) RSA：每个宿都沿最短路径接入主干

时延排序： SPT = RSA < RST ≈ ST < MST
RSA 只有一个 Steiner 点，路径更直

一棵树 = 一个有序段列表：把“树”编码进报文头

= 树上的一条边 = 一个标签；整棵树按 BFS

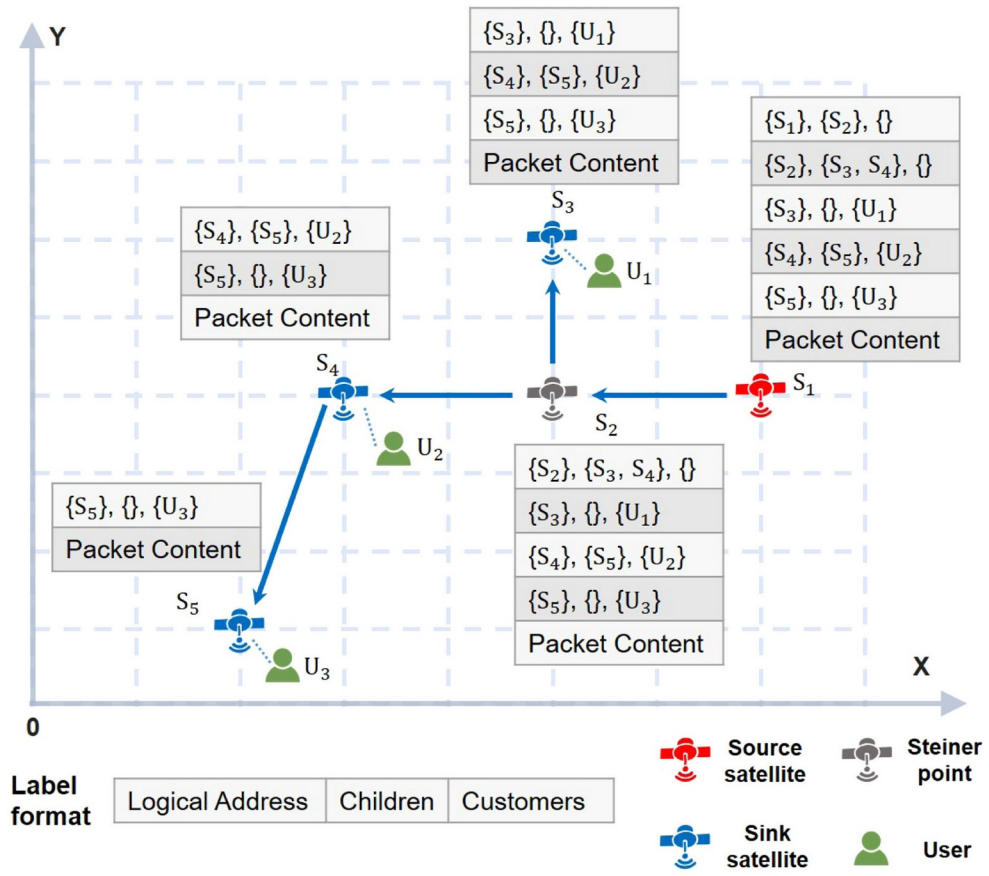


Fig. 7 4 条边 / 4 段的组播树与标签栈状态：BFS 序列 (S1–S5)，各成员卫星报文头不同

为什么用 BFS 排序

BFS 从根逐层展开，与标签栈“先入先出”的读写次序天然一致

副本在分叉处复制栈，子节点只需读取自己的活动标签

标签的三个字段（Fig. 7 的 Label format）

① 本地目的地址 — 子节点的逻辑地址：本段的本地终点

② 子节点集合 — 变长，含结束标记；用于派生下一段

③ 附着用户集合 — 变长，含结束标记；用于最终交付

两条边界规则

Steiner 点：无附着用户 ⇒ 第三字段只有结束标记

宿卫星：无子节点 ⇒ 第二字段只有结束标记

四类卫星、一条判定规则：地址相等即到站，否则继续转发

判定只看一件事：活动标签的目的地址是否等于自身逻辑地址。这让中继卫星完全无状态

1 **控制器** 算出树 → 配置源卫星转发表

2 **源卫星** 把段列表编码为标签栈写入报文头

3 **成员卫星** 读活动标签 → 复制→ 派生子段

4 **接收卫星** 摘除报文头 → 交付附着用户

中继卫星：只做几何转发

目的地址 ≠ 自身地址 ⇒ 我是中继

按段的目的坐标选择端口，就近向目的推进

不记录路径状态、不需要转发表项

成员卫星：本地目的，派生下一段

目的地址 = 自身地址 ⇒ 我是本段终点

读出活动标签中的子节点集合

为每个子节点更新活动标签，多端口复制发出

接收卫星：有用户，摘头交付

若本身附着用户，则它同时是宿卫星

在把报文交给用户前移除报文头

用户看到的是原始组播内容

两类边界情形

Steiner 点：无附着用户 ⇒ 第三字段仅结束标记

宿卫星：无子节点 ⇒ 第二字段仅结束标记

变长字段用结束标记分隔，或用不同取值范围区分

协议收益：无需额外信令协议（控制面简化）· 拓扑变化只需重编程源节点（敏捷）· 中继无需维护状态（可扩展）· 只需标签与自身附着信息即可导包，存储开销低于需要"报文头 + 转发表"双份状态的 BIER 。

段内只保留一个目的坐标：斜边随机走，报文头自然变短

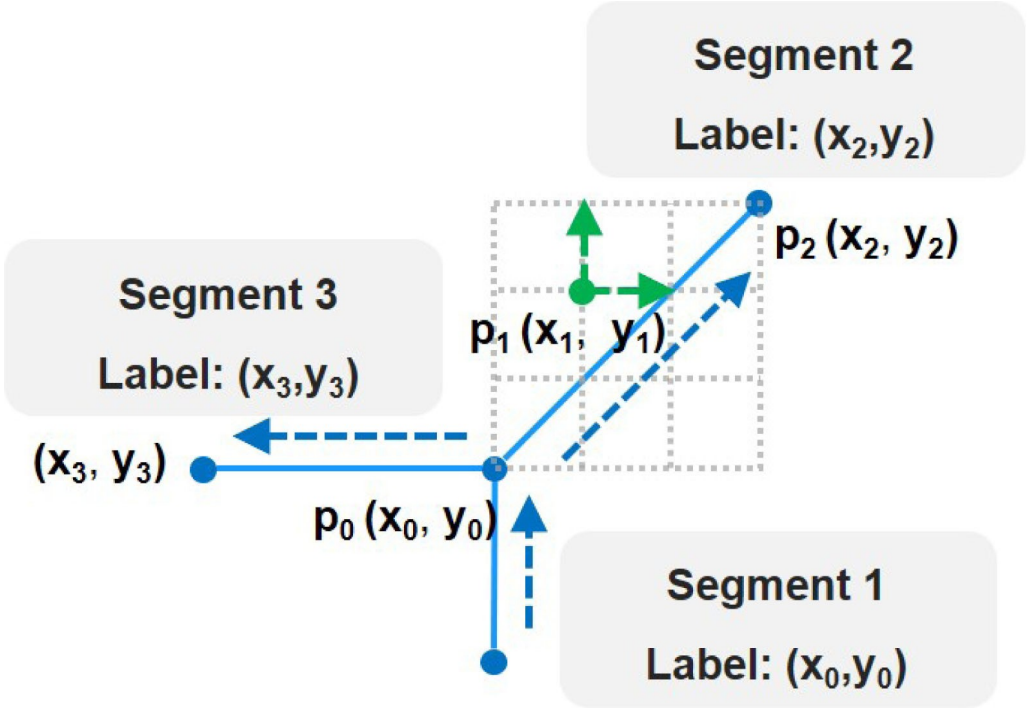


Fig. 8 段内 ECMP：中间节点 p1 可随机选择向上或向右（p0 → p2 的斜边段 2）

论文进一步给出"去标签栈"的扩展：把段存成卫星转发表项，报文头只留 1 个标签，用敏捷性换开销。

段内转发规则（Fig. 8）

垂直段 x1 = x2 ⇒ 下一跳取 (x1, y1+1)

水平段 y1 = y2 ⇒ 下一跳取 (x1+1, y1)（图中段 3）

斜边段 x1 < x2 且 y1 < y2 ⇒ 随机选 (x1+1, y1) 或 (x1, y1+1)

前提 不失一般性设 x0 ≤ x1 ≤ x2 且 y0 ≤ y1 ≤ y2

两个收益

报文头：只需存终点 p2 的坐标，中间节点一律不写 ⇒ 头部显著压缩

负载均衡：报文可在 p0–p2 包围盒内任意等代价路径上散布，缓解热点与拥塞

与本地段路由的分工： ECMP 负责粗粒度随机，本地 SR 负责细粒度指定

段内再分段： d(p0,p2) 条等代价 3- 段路径，任选其一实现细粒度调度

ECMP 负责“粗粒度随机”，本地段路由负责“按需指定”——两者按段组合，就是本文的混合控制策略

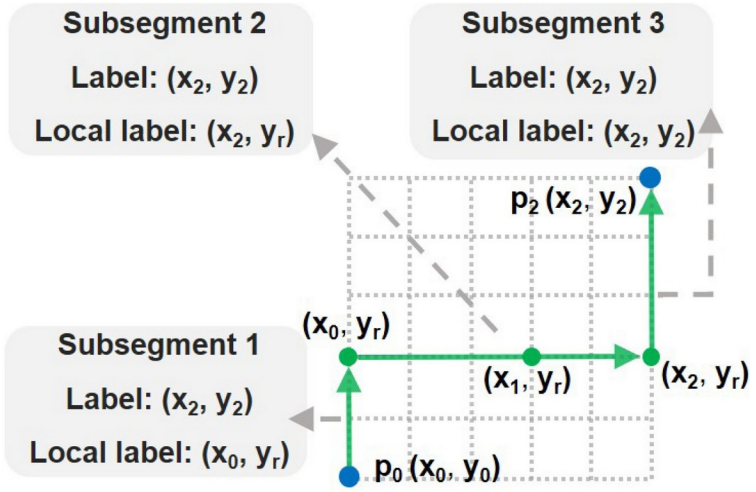


Fig. 9 一个段被切成 3 个子段：本地地标 (x0, yr) 唯一确定一条 3- 段路径

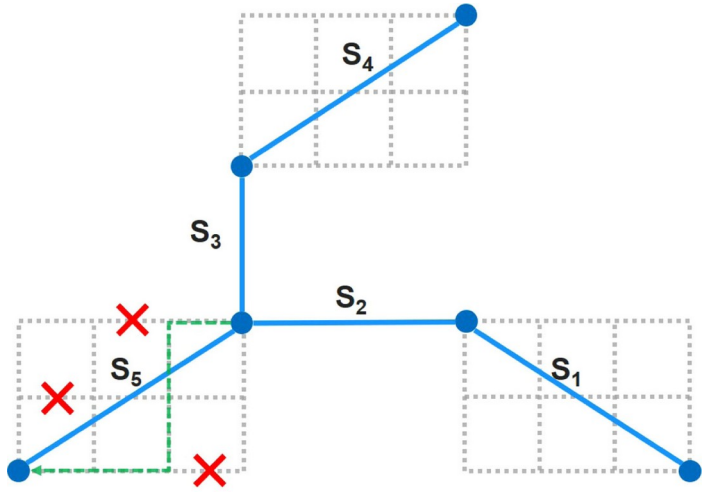


Fig. 10 S1–S4 用 ECMP 感知路由， S5 用本地段路由（斜边 + 特定负载 / 故障约束）

路径枚举与本地标签

地标 (x0, yr) （ y0 < yr ≤ y2 ）或 (xr, y0) （ x0 < xr ≤ x2 ）各对应唯一一条 3- 段路径

沿途本地标签依次更新： (x0,yr) → (x2,yr) → (x2,y2) ，逐子段推进到 p2

共 d(p0,p2) 条等代价候选，源端可随机选或按规则指定

本地标签只在段内有效，不影响全局标签栈的结构

何时启用：按段配置策略

直边 + 无特殊约束的斜边（ S1–S4 ）⇒ 段内 ECMP ： 省表项、粗粒度均衡

有负载要求或故障的斜边（ S5 ）⇒ 本地段路由：细粒度均衡、绕开失效点

连 3- 段路径都被故障击穿 ⇒ 交给树管理（第 6 节）兜底

控制器可对每条边设阈值，按需触发细粒度均衡

标签栈模式 vs 精简转发表模式：同一棵树的两种落地方式

论文把“标签放在报文头”与“标签放进卫星转发表”作为一对权衡，而不是唯一解

对比维度	标签栈模式（label stack）	精简转发表模式（concise table）
报文头内容	完整的有序段列表（多标签栈）	只保留 1 个标签
转发表位置	仅源卫星需要存整棵树	树上的每颗成员卫星各存 1 条表项
表项字段	—（树信息在报文头里）	{组 ID，子节点，附着用户}，后两者变长
控制面	无需额外信令协议，只编程源节点	控制器逐时隙预算表项并分发到各卫星
敏捷性	高：拓扑变化只需重配源卫星	较低：需同步更新多颗卫星的表项
开销特征	报文头大、星上存储省	报文头小、星上表项多

取舍的本质：把状态放在“报文”里（无状态转发、头部开销大）还是放在“卫星”里（头部极小、但要维护表项）。论文默认使用标签栈模式，并对精简模式给出了完整的表项格式与时隙分发流程；精简模式下报文头仅剩 1 个标签（组 ID + 子节点 + 用户），成员卫星 O(1) 查表。

成员变化与故障都会让树“退化”，恢复的本质是重连子树

论文把“成员更新”和“故障处理”统一成同一个问题：断开若干子树的重新连接

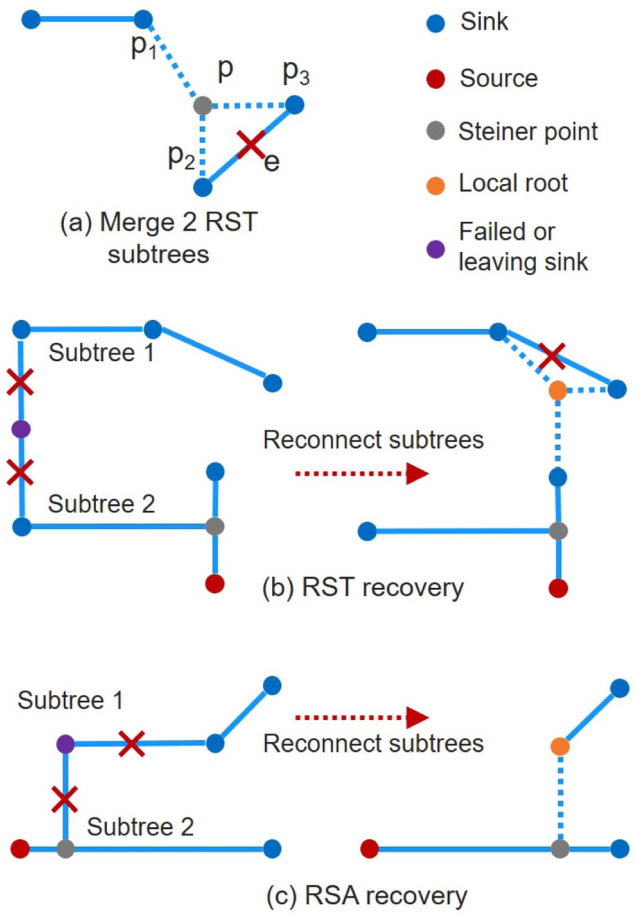


Fig. 11 (a) 用 Steiner 点合并两棵子树；(b) RST 恢复；(c) RSA 恢复（橙 = 本地根，紫 = 失效 / 离组宿）

触发条件

成员离开：控制器删除离开终端、相邻 Steiner 点及相邻边 ⇒ 子树断开 ⇒ 触发恢复并更新源端标签栈；故障同理

RST 恢复：边替换

新增 Steiner 点 p，加边 (p,p1)、(p,p2)、(p,p3)，删除原边 e

两条新边 (p,p2)+(p,p3) 的代价恰等于 e，故该操作代价 = d(p, p1)

对所有“点–边对”算代价，选最小者合并；邻域由生成图定义

迭代至所有子树连成一棵；复杂度 O(log n) [39]

RSA 恢复：最小覆盖

每棵子树的根都是“最靠近源”的 local root

对全部 local root 执行最小覆盖，把它们连到源

只处理根节点 ⇒ 复杂度 O(log n) [42]

最小覆盖可处理障碍 ⇒ 失效点 / 边自然被绕开

恢复完成后，从各 local root 出发自顶向下更新父子关系，并重新生成源端的标签栈——树的最优性由此得到保持。

值得注意：新增成员等价于“只含一个节点的子树”，因此加入 / 离开 / 故障在算法层面是同一套代码路径——这也是把它写成“树管理”而非三个独立模块的原因。

只向“丢包的子树”重发： local root 把重传从全网收敛到局部

此前工作的做法是把丢失报文重发给全体成员——源会被 ACK 与重传流量压垮，这正是本文要修的病

问题：链路 / 节点故障导致丢包，若源把丢失报文重传给所有组成员，源端将同时承受大量 ACK 与重传流量（文献 [12] 的做法不可扩展）。

1 **步骤 1** 源以单播把失败报文发给各 local root，并裁剪冗余标签栈（受影响节点已收到过，无需重发）

2 **步骤 2** 到达 local root 后，按正常组播在子树内重传，只覆盖真正丢包的节点

local root 如何确定

树恢复时若某子树被重新接到源上，另一棵未接上的子树即标记为“待重路由”

其被接入的节点（或新 Steiner 点）就是该子树的 local root

RSA 中：若某 local root 经由另一 local root 连到源，则它不再被视为 local root

多故障同时发生

方案 A：从源到各 local root 建一棵“临时树”，把标签栈相应扩展（代价是头部更大）

方案 B：退化为沿整棵树重发失败报文（实现最简，开销最大）

收益与代价

偶然单点故障时，单播到 local root 几乎不浪费流量

只有多故障并发才需要额外路由信息，属于可控的退化路径

量化：组规模 ⩽ 160 时单次加入 / 离开 / 故障处理仅需 0.01–1 ms；典型情形下每次故障只产生 1 个 local root，这也是局部重传能显著省流量的根本原因。

第四部分

实验评估

把“公平比较”写进参数：同一拓扑、同一分布、同一随机种子集合

所有结论都建立在 100 次独立试验的平均值上，且每次只扫描一个变量、其余保持默认

维度	设置
评价指标	树长（= ，，，，ECMP 承载比例
对比算法	SPT （IPM）、 MST ，ST ，BBSRT ，RST ，RSA
试验次数	每组结果取 100 次独立试验的平均值
网络规模	M×M 栅格，默认 M = 50 ； 单独扫描 M ∈ [20, 70]
组规模	默认 80 ； 单独扫描 n ∈ {10, 20, 40, 80, 160, 320}
成员分布	均匀分布 + 两种非均匀分布（ 50% / 100% 聚集）

六种算法各自代表什么

SPT = 传统 IP 组播的代表； MST / ST / BBSRT = 经典树算法基线

三种成员分布怎么造

所有节点先均匀分布；随机选 $\log_2(0.2n)$ 个节点为核心节点

再把 50% 或 100% 的非核心节点迁入其最近核心节点的包围盒

动机：现实中人口聚集在城市，成员不会均匀分布

默认参数（ M = 50 ，n = 80 ）只用于扫描单个变量时固定其余条件，不代表最优配置。

5
评价指标

6
对比算法

3
成员分布

100
次试验取平均

2
类动态事件（成员 / 故障）

RST 全面最优；IPM 所用的 SPT 比它差 50%–100%

树长直接对应带宽消耗，这是本文最重要的性能论据：换一棵树，就能省一半以上的 ISL 带宽

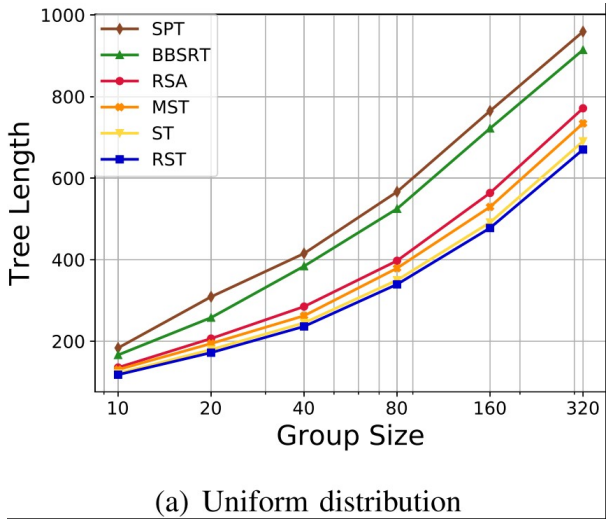


Fig. 12(a) 均匀分布：树长 vs 组规模

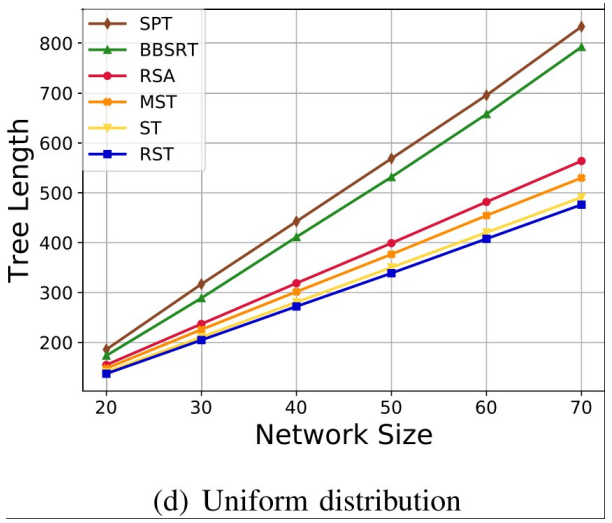


Fig. 12(d) 均匀分布：树长 vs 网络规模

最优
RST 树长最小 → 最省带宽

+3%–5%
ST 次优，但与 RST 的差距仅 3%–5%

+10%
MST 平均比 RST 差约 10%

+10%–20%
RSA 比 RST 差 10%–20%（换来最短时延）

+50%–100%
SPT / BBSRT 比 RST 差 50%–100%

- **横向** SPT 与其他算法的差距显著拉大：成员越多，链路重叠机会越多，其余算法更能省
- **纵向** 各算法之间的差距基本稳定 ⇒ 选树算法时网络规模影响很小

RSA 把时延钉在地理下界，且随组规模增大保持平稳

读图要点：图中刻意省略了 SPT——RSA 同为最短路径树，时延最优；真正的对比是 RSA RST / ST / MST

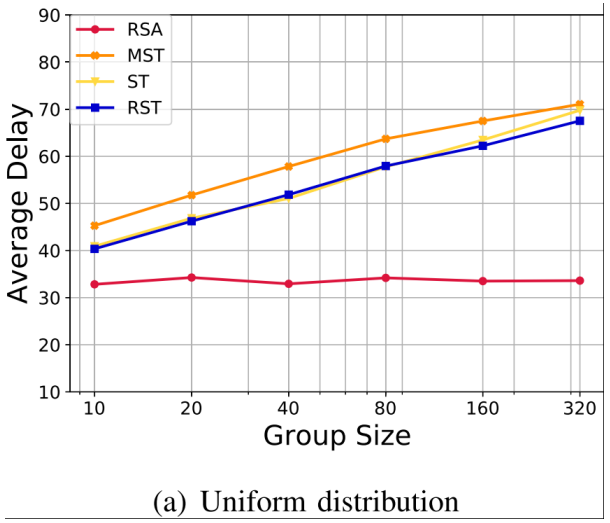


Fig. 13(a) 均匀分布：平均时延 vs 组规模

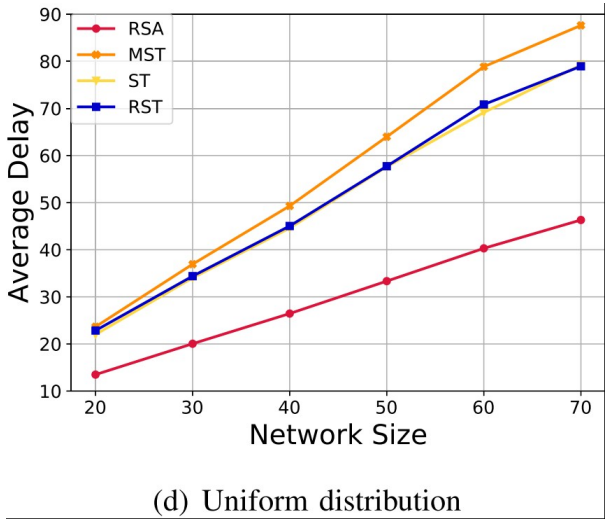


Fig. 13(d) 均匀分布：平均时延 vs 网络规模

- **最强基线** RSA 与 SPT 都产生最短路径树 ⇒ 时延最小，故图中省略 SPT
- **RSA 平稳** 组规模增大时 RSA 时延几乎不变； RST / ST / MST 则明显恶化
- **RST ≈ ST** 两者时延相近，但都明显优于 MST
- **网络规模** RST 与 RSA 的差距随 M 增大基本稳定 ⇒ 选树时网络规模对时延影响也小
- **分布影响** 聚集分布下 RST 时延最小；组规模较大时，均匀 vs 100% 聚集的差距可达 25%
- **结论** 大组播组场景下， RSA 在"省带宽 + 低时延"之间取得最佳平衡

为什么 RSA 时延"钉得住"： RSA 的每条源–宿路径都是直角最短路，树长再优化也不允许任何一条路径变长；而 RST / MST 为省总长会"绕路拼接"，宿越多绕得越多，故时延随组规模恶化。

O(n log n) ，O(log n) 恢复：足以支撑“逐时隙重建”

卫星网络必须周期性重构组播树；如果建树要几十毫秒，协议就根本无法落地——这张图回答了可行性问题

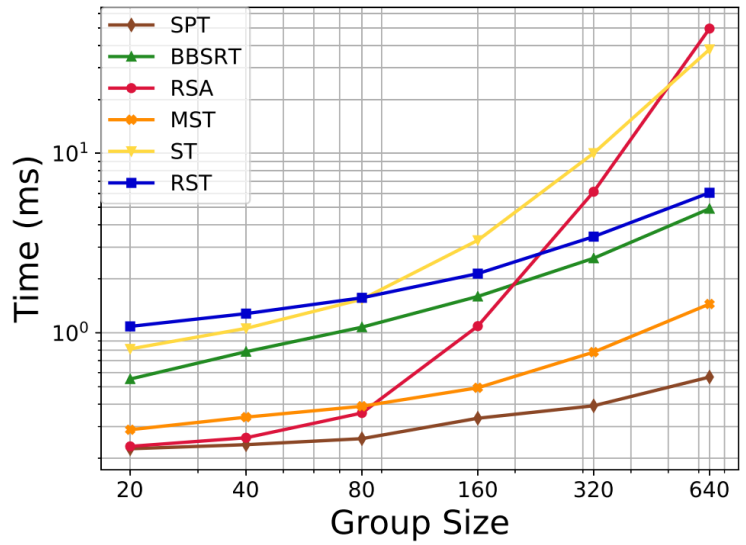


Fig. 14 算法运行时间 vs 组规模（纵轴对数刻度）

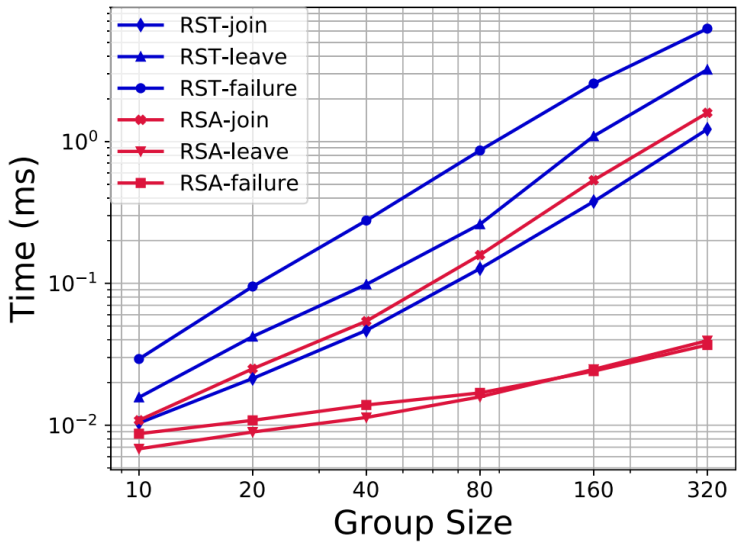


Fig. 15 组管理与树恢复的运行时间（RST / RSA 的 join / leave / failure ）

建树：几百个成员也只要几毫秒

所选 O(n log n) 算法 [39]，[42] 在数百成员规模下仍能在数毫秒内建出 RST / RSA 树

相对 SPT / ST 的绝对时间优势不重要，关键是“毫秒级”足以匹配切换与成员更新节奏

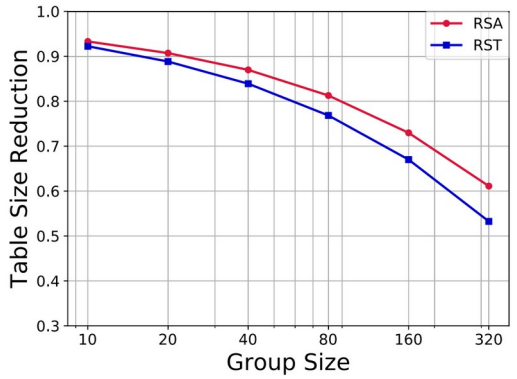
恢复：单次操作 0.01–1 ms

组规模 ≤ 160 时，加入 / 离开 / 故障处理单次仅需约 0.01–1 ms

RST-join 略慢于 RSA-join ；故障处理（ failure ）低于 join / leave ，恢复开销可忽略

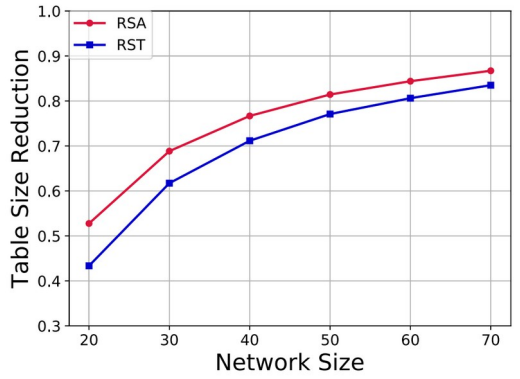
省 90% 表项、 90% 流量走 ECMP： 段路由的两个量化收益

这两张图回答的是“协议本身值不值”：转发表省了多少、有多少流量真正走上了等代价多路径



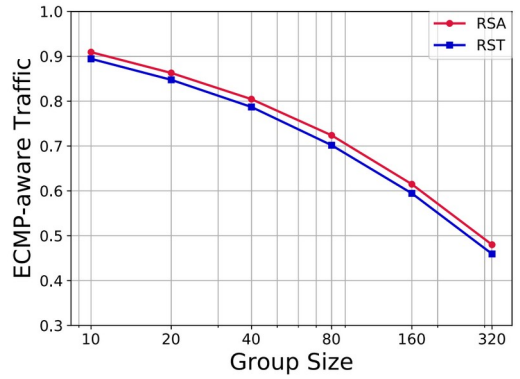
(a) group size

Fig. 16(a) 表项压缩率 vs 组规模



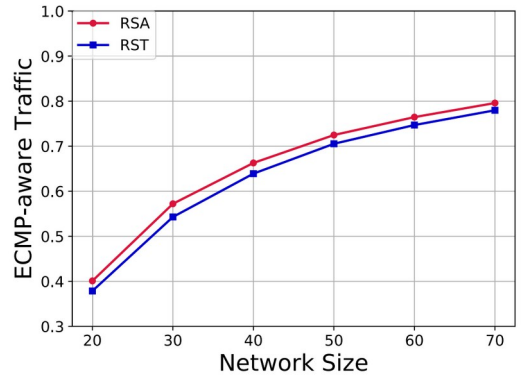
(b) network size

Fig. 16(b) 表项压缩率 vs 网络规模



(a) group size

Fig. 17(a) ECMP 承载比例 vs 组规模



(b) network size

Fig. 17(b) ECMP 承载比例 vs 网络规模

转发表压缩（ Fig. 16 ）

组规模 10 时最高可省 90% 表项；组越稀疏、网络越大，省得越多

RST 比 RSA 略多省一点：树更短 ⇒ Steiner 点更少 ⇒ 需要表项的节点更少

ECMP 承载比例（ Fig. 17 ）

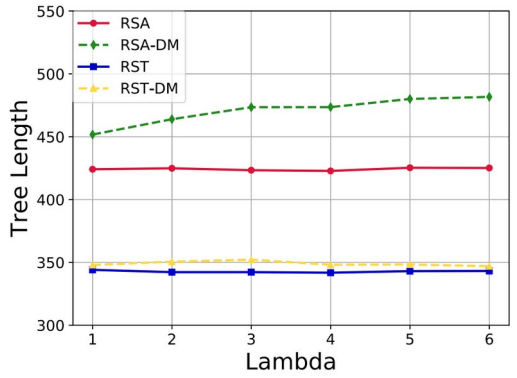
组规模小时最多约 90% 流量经 ECMP 转发；组稀疏 / 网络大时斜边更多

这正是“随机斜边”设计的直接体现：流量被摊到包围盒内的多条等代价路径上

两项收益同源：段内只保留目的坐标，既不必在报文头里写中间节点（省表项），也允许报文在包围盒内自由选路（用 ECMP ）。

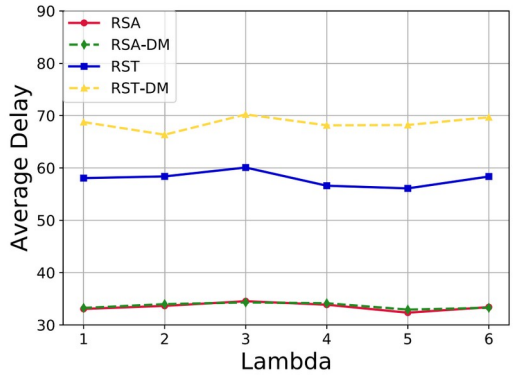
衰减很轻：动态事件下树长几乎不动，时延劣化可控

成员按均值 1/λ 的指数分布加入 / 离开；故障同分布发生。图中 DM = 动态成员， BU = 故障备份路径



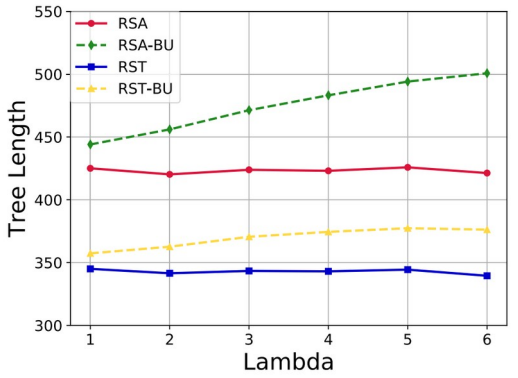
(a) Tree length

Fig. 18(a) 动态成员：树长



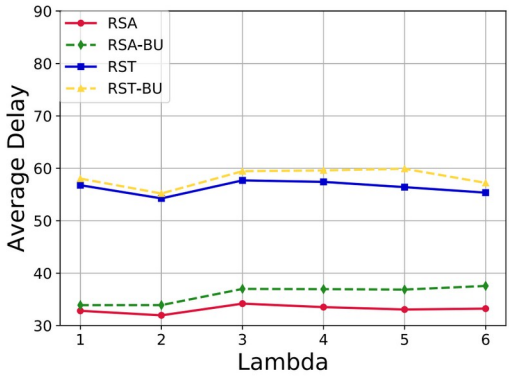
(b) Average delay

Fig. 18(b) 动态成员：平均时延



(a) Tree length

Fig. 19(a) 故障：树长



(b) Average delay

Fig. 19(b) 故障：平均时延

动态成员（Fig. 18）

RST-DM / RSA-DM 在树长与平均时延上都相当平稳，说明加入 / 离开的恢复策略有效

代价：动态性可使 RSA 的时延劣化最多约 30%，进一步优化 join/leave 会带来更高时间复杂度

故障（Fig. 19）

RST-BU / RSA-BU 曲线平稳，树长与时延的劣化都很轻微

这直接验证了容错模块（树恢复 + 局部重传）的有效性：故障不引发性能断崖

注意读图方式：实线是理想静态下的树（RST / RSA），虚线是叠加动态事件后的结果；两条线越贴近，说明自愈机制越“无感”。

选树准则：带宽优先 RST，时延优先 RSA

论文没有给出“唯一最优算法”，而是给出“按目标选树”的工程准则——这比单一排名更有价值

应用目标	推荐算法	依据（实验结论）	相对收益量级
最小化带宽 / ISL 负载	RST	树长最小；组规模增大时优势更明显	比 SPT 省 50%–100%，比 MST 省 ~10%
带宽 + 时延双目标	RSA	最短路径约束下树长最小，时延贴近下界	树长仅比 RST 差 10%–20%，时延最优
依赖传统 IP 组播栈	不推荐 SPT	树长为 RST 的 1.5–2 倍，带宽代价过高	无收益，仅兼容性
星上 TCAM 紧张	段路由（精简模式）	组规模 10 时最高省 90% 表项	表项减少最高 90%
ISL 热点 / 拥塞	段内 ECMP + 本地 SR	小规模组最多 90% 流量经 ECMP 承载	流量在包围盒内自动摊平

一个容易被忽略的结论：网络规模 M 对树长与时延的影响都很小，因此算法选择不必随星座规模重新调参——这对长期运行的星座很关键。

第五部分

总结与思考

四条贡献，四条被实验支持的结论

论文的结论颗粒度很细：它不宣称“全面最优”，而是明确指出不同目标下该选哪棵树

C1 SDM 框架

用 RST 与 RSA 构建组播树，替代 IPM 的 SPT

RST 树长最优， SPT 比它差 50%–100%； RSA 在树长仅劣 10%–20% 的前提下取得最优时延

C2 段路由协议

树 = 有序段列表；段内 ECMP；切换由逻辑地址吸收

小规模组最高省 90% 表项，最多 90% 流量经 ECMP 承载，中继卫星无需路径状态

C3 树恢复

RST 边替换 / RSA 最小覆盖，统一处理成员更新与故障

O(log n) 复杂度；组规模 ⩽ 160 时单次操作 0.01–1 ms

C4 局部重传

local root 定位丢包子树，只对该子树重传

动态成员与故障下树长 / 时延劣化轻微； RSA 时延劣化上限约 30%

总体判断：本文的核心价值不在“发明一种新树”，而在“证明栅格拓扑 + 段路由能把已有树算法的收益真正兑现到卫星网络上”。

三个可以迁移到其他时变网络的设计思想

拓扑即算法

- 把“卫星会动”转化为“栅格不变”：树在逻辑栅格上算，物理拓扑再映射回来
- 于是几十年的直角斯坦纳树理论可以直接复用，无需为卫星网络重造算法
- 代价：依赖时隙内拓扑近似静态，且需要精确的轨道先验

地址即状态

- 用户与卫星共享地标逻辑地址，切换只改指针、不改树
- 路由状态被“外置”到地址体系里，控制面因此极简（只编程源卫星）
- 这解释了为什么它的表项开销能比 BIER 更低

分层自愈

- 粗粒度用 ECMP 随机散布，细粒度用本地段路由指定路径，兜底用树恢复
- 三层机制对应三类问题，且都不需要全局重算
- 工程上很实用：可按段的 SLA 选择策略

为什么这些思想值得关注

它们不是针对卫星的“专用技巧”，而是把“动态拓扑”抽象成“静态图 + 地址映射”的通用范式，可迁移到无人机网络、低轨 IoT 星座、甚至数据中心的光路调度。

反过来说，凡是“节点会动、但相对位置关系稳定”的网络，都可以照搬这套三段式：抽象 → 建树 → 段路由。

评价：方法的优雅之处在于“少发明、多重用”——把成熟的直角斯坦纳树与段路由组合，解决一个此前没人组合过的场景。

四个可以继续做的方向：从仿真假设走向真实星座

仿真规模与真实性

M ≤ 70、组规模 ⩽ 640 的抽象栅格仿真，未接入真实星历与链路预算

未量化激光 / 微波 ISL 的带宽与误码差异，等带宽假设偏理想

强假设依赖

依赖星–用户同步时钟与"逐时隙拓扑静态"近似

缝两侧两个独立栅格需地面骨干互联，跨网格组播未展开

对比基线不足

与 BIER、Tree-SID 等 SR 组播方案只有定性比较，缺少同场景定量对照

未与文献 [32]、[33] 的 LEO SDM 树构建工作直接同图对比

优化目标单一

以树长 / 时延为主，未纳入星上能耗、链路利用率方差、队列时延等

故障模型是随机指数分布，缺少对区域性中断（如极区、天气）的建模

可延伸的工作

把能耗与链路异构纳入树构建目标 · 用真实 Walker 星历做时隙级验证 · 与 BIER / Tree-SID 做同场景对照 · 研究跨缝双栅格的联合组播 · 把失败包重传与组播拥塞控制联合设计